

RAPORT PROJEKTOWY

API USGS Earthquake

Wprowadzenie do baz danych

Andżelika Zuba, Agata Wójcik, Stanisław Zieliński

1. Opis projektu

Celem projektu było zaprojektowanie relacyjnej bazy danych i zbudowanie kompletnego przepływu danych od publicznego API do warstwy analitycznej i wizualizacyjnej. Jako źródło wybrano USGS Earthquake Catalog.

Zrealizowano wymagania:

- 4 tabele merytoryczne + 1 słownikowa + 1 techniczna (logowanie importu),
- zapis historyczny i ochrona przed duplikacją rekordów,
- automatyczny, cykliczny import danych w konfigurowalnym interwale,
- 8 zapytań SQL użytych do analiz i raportów,
- 3 wizualizacje i 15 filtrów.

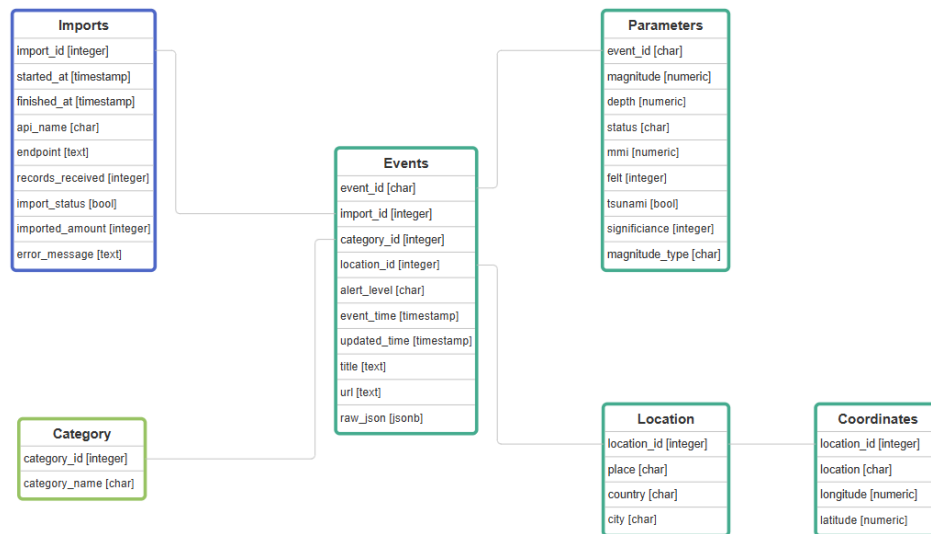
2. Struktura bazy danych

Schemat bazy danych zawiera 6 tabel pogrupowanych w trzy warstwy: techniczna, słownikowa i merytoryczna (Tab. 1).

Tab. 1. Opis tabel w bazie danych.

Nazwa	Typ	Opis
imports	techniczna	Log każdego importu: czas startu/końca, status, liczba rekordów, komunikat błędu.
category	słownikowa	Słownik kategorii zdarzeń.
location	merytoryczna	Miejsce zdarzenia: pełna nazwa, miasto, kraj.
coordinates	merytoryczna	Współrzędne geograficzne zdarzenia.
events	merytoryczna	Zdarzenie: klucz event_id, czas, alert, URL, surowy JSON.
parameters	merytoryczna	Parametry fizyczne zdarzenia: magnituda, głębokość, tsunami, MMI, istotność.

Do każdego zdarzenia przypisany jest oryginalny wpis ze strony w celach zweryfikowania poprawności danych. Cała struktura bazy widoczna jest na Rys. 1.



Rysunek 1. Schemat bazy danych.

3. Tworzenie bazy, import i archiwizacja danych

3.1. Tworzenie bazy danych

Tworzenie bazy danych składa się z następujących plików:

1. schema.sql - zawiera skrypt SQL definiujący strukturę bazy danych, w tym tabele, klucze oraz indeksy.
2. config.py - plik konfiguracyjny zawierający parametry połączenia z bazą danych, adres URL API oraz ustawienia importu danych.
3. create_database.py - tworzy bazę danych PostgreSQL (sprawdza przy okazji czy baza już nie została wcześniej utworzona).
4. create_tables.py - Wykonuje schema.sql w docelowej bazie danych.
5. Setup.py - Punkt wejścia instalacji. Uruchamia kolejne etapy tworzenia bazy danych.

3.2. Architektura importu

Moduł import_data.py realizuje kompletny cykl import - zapis:

6. Logowanie startu - tworzy rekord w tabeli imports ze statusem FALSE i czasem startu.
7. Pobranie danych z API – wykonuje zapytanie do USGS Earthquake API z parametrami.
8. Parsowanie i zapis - dla każdego zdarzenia: upsert kategorii, lokalizacji, współrzędnych, zdarzenia i parametrów.
9. Aktualizacja logu - update rekordu imports (status TRUE, liczba importowanych, czas zakończenia).
10. Obsługa błędów - w przypadku wyjątku zapisuje error_message w logu importu i ponownie zgłasza wyjątek.

3.3. Ochrona przed duplikacją

Każda operacja *insert* używa klauzuli *ON CONFLICT DO UPDATE*. Dzięki temu ponowny import tych samych zdarzeń z API w przypadku zachodzących na siebie okien czasowych nie tworzy duplikatów, lecz aktualizuje dane do najnowszej wersji z API.

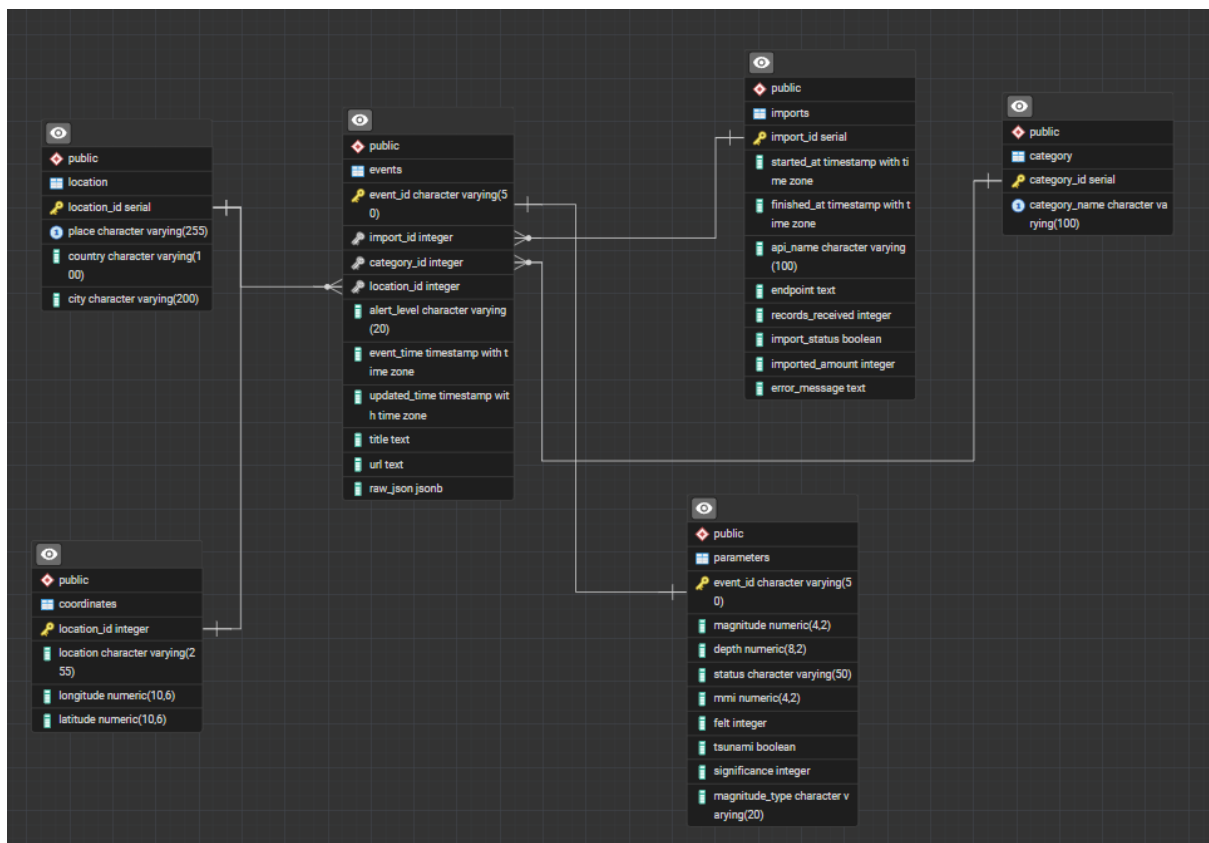
3.4. Import cykliczny

Flaga *--loop* uruchamia pętlę, która wywołuje import co dany interwał (domyślnie 60). Każda iteracja obejmuje pewne okno godzin wstecz (domyślnie 3 h), co zapewnia ciągłość zapisu nawet w razie chwilowej niedostępności API.

```
PS C:\Users\andze\Desktop\bzdety\postgrespostgres\projekt\Projekt_bazy_danych> python import_data.py --hours 24
Import zakończony. Przetworzono zdarzen: 51
PS C:\Users\andze\Desktop\bzdety\postgrespostgres\projekt\Projekt_bazy_danych> python import_data.py --loop
Import zakończony. Przetworzono zdarzen: 9
Następny import za 30 minut.
Import zakończony. Przetworzono zdarzen: 8

.exe c:/Users/andze/Desktop/bzdety/postgrespostgres/projekt/Projekt_bazy_danych/setup.py
Utworzono baze danych 'earthquakes'.
Utworzono lub zaktualizowano tabele.

PS C:\Users\andze\Desktop\bzdety\postgrespostgres\projekt\Projekt_bazy_danych> python setup.py
Baza danych 'earthquakes' już istnieje.
Utworzono lub zaktualizowano tabele.
```



4. Zapytania SQL i analiza danych

Warstwa analityczna zawiera dwa rodzaje zapytań: dynamiczne filtrowanie oraz predefiniowane raporty. Łącznie zaimplementowano 8 zapytań raportowych i 15 filtrów. Różne typy raportów przyjmowane są jako parametry funkcji, przy czym wywołanie funkcji *report* bez dodatkowych

argumentów tworzy wszystkie raporty i je wypisuje. Całość jest skryptem pythonowym z wynikami wypisywanymi w terminalu.

Tab. 2. Dostępne raporty do wygenerowania wraz z opisem.

Nazwa raportu	Opis
summary	Ogólne podsumowanie bazy.
top	Top 10 najsilniejszych trzęsień.
by_category	Statystyki wg kategorii zdarzenia.
by_country	Top 15 krajów wg liczby zdarzeń.
by_day	Liczba zdarzeń w podziale na dni.
magnitude_dist	Rozkład wg przedziału magnitudy.
tsunami	Zdarzenia z ostrzeżeniem tsunami.
imports	Log importów (tabela techniczna).

Funkcja `build_filter_query` buduje zapytanie SQL dynamicznie na podstawie argumentów CLI. Obsługuje 15 filtrów (Tab. 3). Dodatkowo zaimplementowano parametry sortowania:

--order-by (time/magnitude/depth/significance),
 --asc (domyślnie malejąco),
 --limit (domyślnie 50).

Tab. 3. Dostępne argumenty do funkcji filtrującej.

Nr	Filtr CLI	Warunek SQL	Typ
1	--min-magnitude	p.magnitude >= ?	numeryczny
2	--max-magnitude	p.magnitude <= ?	numeryczny
3	--min-depth	p.depth >= ?	numeryczny
4	--max-depth	p.depth <= ?	numeryczny
5	--from	e.event_time >= ?	data
6	--to	e.event_time <= ?	data
7	--category	c.category_name = ?	tekstowy (dokładny)
8	--country	l.country ILIKE '%?%'	tekstowy (częściowy)
9	--city	l.city ILIKE '%?%'	tekstowy (częściowy)
10	--place	l.place ILIKE '%?%'	tekstowy (częściowy)
11	--alert	e.alert_level = ?	tekstowy (dokładny)
12	--magnitude-type	p.magnitude_type = ?	tekstowy (dokładny)
13	--status	p.status = ?	tekstowy (dokładny)
14	--min-significance	p.significance >= ?	numeryczny
15	--tsunami	p.tsunami = TRUE	boolean (flaga)

```
PS C:\Users\andze\Desktop\bzdety\postgrespostgres\projekt\Projekt_bazy_danych> python analyze_data.py report

=== Podsumowanie ogolne ===
liczba_zdarzen | srednia_mag | max_mag | min_mag | max_glebokosc
-----+-----+-----+-----+-----
973           | 3.80       | 7.80   | 2.50   | 617.75

Wierszy: 1

=== Top 10 najsilniejszych trzesien ziemi ===
event_time | magnitude | place | country | depth
-----+-----+-----+-----+-----
2026-06-08 01:37:41.803000+02:00 | 7.80 | 26 km SW of Kablalan, Philippines | Philippines | 55.19
2026-06-16 05:27:44.418000+02:00 | 6.70 | 43 km ESE of Palu, Indonesia | Indonesia | 10.00
2026-06-08 02:55:11.958000+02:00 | 6.50 | 20 km WSW of Balangonan, Philippines | Philippines | 69.00
2026-06-16 11:06:55.226000+02:00 | 6.30 | 260 km SSE of Dunhuang, China | China | 10.00
2026-06-15 11:18:38.846000+02:00 | 6.20 | 67 km ESE of Pondaguitan, Philippines | Philippines | 111.92
2026-06-02 00:12:36.092000+02:00 | 6.20 | 23 km WSW of San Lucido, Italy | Italy | 243.00
2026-06-07 12:41:58.596000+02:00 | 6.10 | 124 km SE of Severo-Kuril'sk, Russia | Russia | 35.00
2026-06-08 20:00:27.508000+02:00 | 6.10 | 102 km WNW of Mantua, Cuba | Cuba | 26.00
2026-06-10 02:44:21.127000+02:00 | 6.00 | Auckland Islands, New Zealand region | New Zealand region | 13.67
2026-06-08 01:49:14.512000+02:00 | 6.00 | 6 km SSW of Pangyan, Philippines | Philippines | 42.91

Wierszy: 10

=== Liczba zdarzen wg kategorii (tabela slownikowa) ===
category_name | liczba_zdarzen | srednia_mag
-----+-----+-----
earthquake | 973 | 3.80

Wierszy: 1

PS C:\Users\andze\Desktop\bzdety\postgrespostgres\projekt\Projekt_bazy_danych> python analyze_data.py filter --min-depth 120 --from 2026-06-15

event_id | event_time | magnitude | depth | category | place | country | alert | tsunami | sign
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----
us7000stm0 | 2026-06-17 12:51:00.310000+02:00 | 4.70 | 163.97 | earthquake | 1 km SE of Villanueva, Colombia | Colombia | | False | 340
us7000stk2 | 2026-06-17 05:09:12.809000+02:00 | 4.40 | 246.62 | earthquake | 298 km SW of Houma, Tonga | Tonga | | False | 298
us7000stbe | 2026-06-16 09:09:26.884000+02:00 | 4.30 | 604.53 | earthquake | Fiji region | | | False | 284
us7000st8e | 2026-06-16 02:00:25.795000+02:00 | 4.30 | 137.92 | earthquake | 33 km SW of Ollagüe, Chile | Chile | | False | 284
aka20261ulguh | 2026-06-15 23:49:01.840000+02:00 | 3.00 | 126.00 | earthquake | 38 km N of Nome, Alaska | Alaska | | False | 138
aka20261ucmmd | 2026-06-15 19:25:42.618000+02:00 | 2.50 | 133.30 | earthquake | 66 km ESE of Denali National Park, Alaska | Alaska | | False | 96
us7000st3p | 2026-06-15 15:03:41.426000+02:00 | 4.40 | 137.70 | earthquake | 36 km NE of Calama, Chile | Chile | | False | 298
us7000st0w | 2026-06-15 10:00:17.147000+02:00 | 4.30 | 590.34 | earthquake | 284 km E of Levuka, Fiji | Fiji | | False | 284
us7000st0n | 2026-06-15 08:58:32.374000+02:00 | 4.50 | 172.57 | earthquake | 34 km NNE of Komodo, Indonesia | Indonesia | | False | 312
us7000st08 | 2026-06-15 07:24:51.203000+02:00 | 4.50 | 184.37 | earthquake | 123 km SE of Ollagüe, Chile | Chile | | False | 312
us7000ssyx | 2026-06-15 02:59:36.840000+02:00 | 4.40 | 168.73 | earthquake | 67 km WSW of Paratunka, Russia | Russia | | False | 298
```

5. Wizualizacje danych

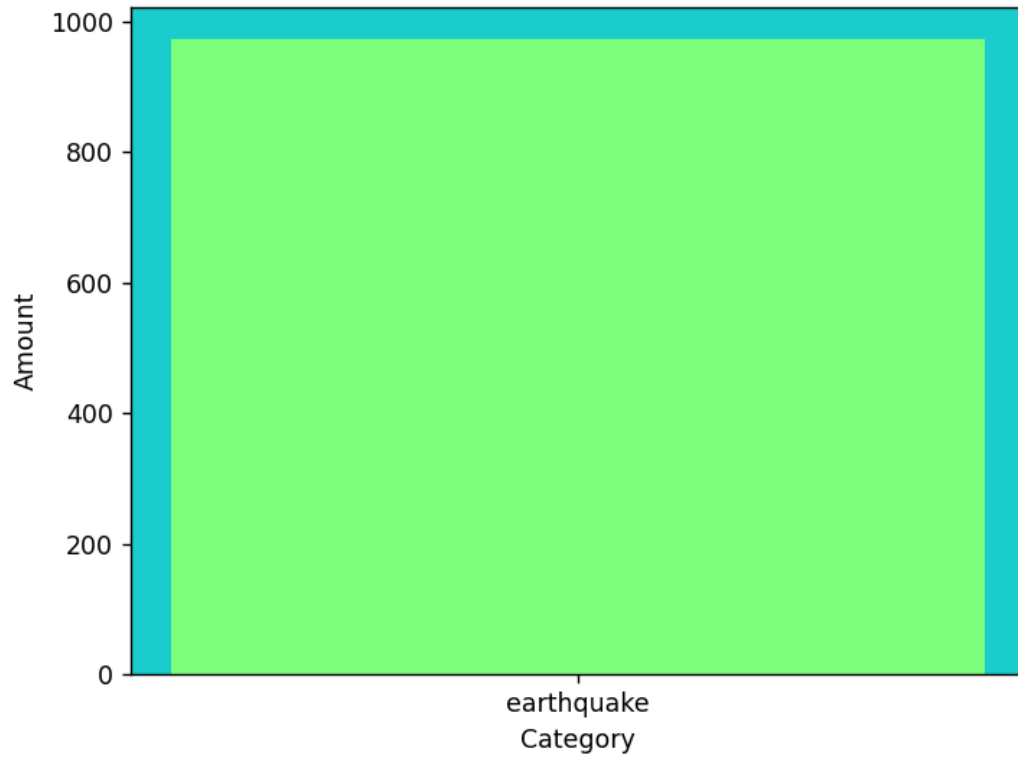
Moduł visualize_data.py implementuje 3 wizualizacje uruchamiane z poziomu CLI poleceniem python visualize_data.py visualize --type N.

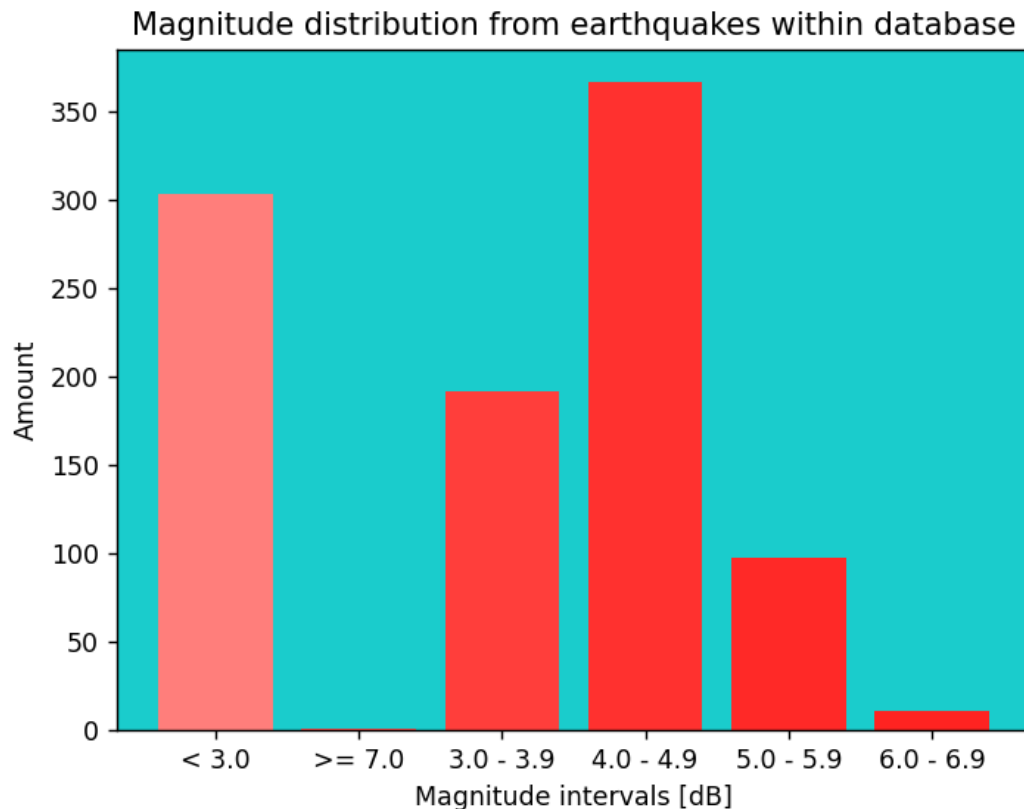
Typ	Nazwa raportu	Rodzaj wykresu
1	top 10 trzęsień (mapa)	Mapa świata z zaznaczonymi obszarami wystąpień zdarzeń
2	by_category (kategorie)	Wykres słupkowy z kolorowaniem gradientowym
3	magnitude_dist (rozkład)	Wykres słupkowy rozkładu magnitudy

Top 10 most powerful earthquakes within database



Earthquake types divided into categories within database





6. Struktura projektu

Plik	Rola
schema.sql	Skrypt SQL definiujący strukturę bazy danych, w tym tabele, klucze oraz indeksy.
config.py	Plik konfiguracyjny zawierający parametry połączenia z bazą danych, adres URL API oraz ustawienia importu danych.
create_database.py	Tworzy bazę danych PostgreSQL (sprawdza przy okazji czy baza już nie została wcześniej utworzona).
create_tables.py	Wykonuje schema.sql w docelowej bazie danych.
setup.py	Punkt wejścia instalacji - wywołuje create_database i create_tables.
db.py	Fabryka połączeń pycopg2 (używany przez analyze_data i inne moduły).
import_data.py	Pobieranie danych z USGS API, parsowanie GeoJSON, upsert do DB, logowanie.
queries.py	Definicje zapytań SQL: raporty oraz filtry.
cli.py	Definicja interfejsu CLI z argparse: podkomendy filter i report z wszystkimi argumentami.
display.py	Wydruk wyników w konsoli w postaci tabeli.
analyze_data.py	Punkt wejścia warstwy analitycznej - łączy CLI, DB, queries i display.
visualize_data.py	Wizualizacje: mapa świata + wykresy słupkowe.
requirements.txt	Zależności: pycopg2-binary, requests, matplotlib, basemap.
README.md	Instrukcja uruchomienia projektu.

7. Uruchomienie projektu

Do uruchomienia projektu potrzebne są:

- Python 3.10+,
- PostgreSQL 13+ dostępny lokalnie,
- Zależności wypisane w requirements.txt.

Cały program należy uruchomić zgodnie z Tab. 5.

Tab. 5. Kolejność uruchamiania skryptów wraz z opisem.

Krok	Komenda	Opis
1	py setup.py	Tworzy bazę danych earthquakes i tabele wg schema.sql.
2	py import_data.py --hours 24	Jednorazowy import z ostatnich 24 godzin.
2a	py import_data.py --loop	Import cykliczny (co 60 min, okno 3 h).
3	py analyze_data.py report	Wszystkie 8 raportów analitycznych w konsoli.
4	py analyze_data.py filter --min-magnitude 5 --country Japan --limit 20	Przykładowe filtrowanie.
5	py visualize_data.py visualize --type 1	Mapa 10 najsilniejszych trzęsień.

8. Podsumowanie i wnioski

Projekt realizuje pełny przepływ danych: od importu ze źródła zewnętrznego do analizy i wizualizacji. Dzięki samodzielnie zaimplementowanej bazy danych można wyszczególnić ważne dla zespołu informacje bez konieczności zapychania serwera nieprzydatnymi danymi. Taki sposób ułatwia obróbkę danych i zwiększa ich czytelność. Parametryzacja w skryptach dodatkowo pozwala na elastyczny i łatwy import.